

Introduksjon

I denne rapporten beskriver vi arbeidet vårt med Oblig 3, hvor vi har videreutviklet AeroScript, et programmeringsspråk designet for å styre droner autonomt. Denne oppgaven bygger på det vi gjorde i Oblig 2, men her har vi fokusert på å lage en **typekontroller** som kan sjekke om skript skrevet i AeroScript følger språkets regler. Dette er viktig for å sikre at feil i koden blir oppdaget tidlig, før dronen begynner å utføre handlinger.

I tillegg har vi laget flere tester som sjekker at typekontrollen fungerer som den skal. Vi har også sikret at tolken (interpretøren) fortsatt fungerer sammen med typekontrollen, slik at bare korrekt kode blir kjørt.

Hovedmålet med denne oppgaven var altså å lage en robust løsning for typekontroll og sørge for at den dekker alle mulige scenarier i AeroScript.

TypeChecker

TypeChecker-klassen er hjernen i typekontrollen. Den sjekker at alle handlinger i skriptet bruker riktige typer. For eksempel:

- Hvis du skriver `move to point (10, 20)`, sjekker TypeChecker at `(10, 20)` faktisk er en **POINT**.
- Hvis du skriver `ascend by 50`, sjekker den at `50` er et **NUM** (tall).
- Hvis du gjør noe galt, for eksempel `move to 30`, vil TypeChecker kaste en feilmelding som sier at `move to` krever en **POINT**.

Hvordan TypeChecker fungerer:

1. **Oppdeling av skriptet:** Først tar den programmet og deler det opp i mindre deler (ved hjelp av parseren).
2. **Navigasjon:** Den navigerer gjennom hver del (f.eks. `move`, `ascend`, `on message`) og sjekker at typene stemmer.
3. **Feilhåndtering:** Hvis noe er feil, kastes en `TypeError` med en tydelig feilmelding.

JUnit-tester

Testoversikt

Testene ble laget med mål om å dekke både vanlige og edge cases for typeregler i AeroScript. Testene er plassert i `TypeCheckerTest` og dekker blant annet følgende:

- **Gyldige tester:**
 - `move to point (10, 20)`: Testet at `POINT`-typen godtas som et gyldig mål.
 - `ascend by 50`: Testet at `NUM`-typen aksepteres for stigning.
 - `on low battery -> EmergencyLanding`: Testet korrekt bruk av reaksjoner.
- **Ugyldige tester:**
 - `move to 30`: Kaster en `TypeError` fordi `POINT` kreves.
 - `ascend by point (10, 20)`: Kaster en `TypeError` fordi `NUM` kreves for stigning.
 - `on message [] -> ReturnToBase`: Validerer at en gyldig ID må spesifiseres for meldingshendelser.

Utfordringer vi møtte

1. **Integrasjon av typekontroll med eksisterende tolk:** Å kombinere typekontrollen med den eksisterende tolken krevde nøye planlegging for å sikre at de to modulene kommuniserer effektivt.
2. Siden det er eksamensperiode så har det heller ikke vært så lett å planlegge møtene for å jobbe med denne obligen. Mye ble gjort gjennom facetime som var en optimal løsning